

Assignment 4

Kai Yi Ng

Repo: <https://github.com/kaiyi03/CWM-Project/tree/main/assignment4>

For myself:

- Compiler allocate local variables in the stack
 - .text - compiled machine instructions
 - .data - initialised global variables
 - .bss - uninitialised global variables
 - Heap – dynamic memory
 - Stack – local variables or return addresses
-
- .text starts at a fixed address (e.g., 0x400000)
 - Stack starts near top of address space (0x7fff...) and grows downwards
 - Heap grows upward from below .bss

Q2.

```
ubuntu@ubuntu:~/CWM-Project/assignment4/buffer_overflow/4A$ ./main 6
Target Function Ptr: 0x4011dc
```

Q3.

Offset	Observed output	Notes
8	Target Function Ptr: 0x4011dc	Offset too small — return address not reached, program returns normally
24	Target Function Ptr: 0x4011dc Success! Malicious function Called! Segmentation fault (core dumped)	Correct offset — overwrites return address exactly, victim() jumps to steal_password(). Segfault after because steal_password() has no valid return address to go back to
30	Segmentation fault (core dumped)	Offset too large — overwrites return address with 'A' (0x41) garbage, CPU jumps to invalid address
40	Segmentation fault (core dumped)	
60	Segmentation fault (core dumped)	

Q4.

buf starts at the bottom. When you write past it with strcpy, we write upwards through memory. So, to reach the return address, you need to skip past

- 16 bytes of buf
- 8 bytes of saved RBP

Hence 24 bytes total before you reach the return address slot.

```
Higher addresses (toward top of address space)
+-----+
|      return address      | <-- where to go when the function returns
+-----+
|      saved RBP           | <-- caller's base pointer
+-----+
|    local variables       |
+-----+
|      buf[16]             | <-- arrays live here
+-----+
Writing to buf moves stack pointer UPWARDS
```

Q5.

```
ubuntu@ubuntu:~/CWM-Project/assignment4/buffer_overflow/4A$ ./main 5
Target Function Ptr: 0x4011dc
ubuntu@ubuntu:~/CWM-Project/assignment4/buffer_overflow/4A$ ./main 120
Target Function Ptr: 0x4011dc
Segmentation fault (core dumped)
ubuntu@ubuntu:~/CWM-Project/assignment4/buffer_overflow/4A$
```

With a small input, the address is written within buf itself and never reaches the return address, so the program exits normally.

With a large input, strcpy overwrites the return address with 0x41 ('A' character) before reaching the function address, so when victim() returns, the CPU tries to jump to an invalid address and segfaults.

Q6.

Buff Offset	Address	Constant (hex)
Buf[0]	0	ef
Buf[1]	1	be
Buf[2]	2	ad
Buf[3]	3	de

Reversed order.

Q7.

`strcpy(dest, src)` copies bytes from `src` to `dest` until it encounters a null byte `\0`. It does not check whether `dest` is large enough. If `src` is longer than `dest`, `strcpy` keeps incrementing both pointers and writing, overwriting whatever sits above `dest` in the stack frame.

The `strncpy()` function copies the first `n` characters from one string into the memory of another string. Stops copying when the first `n` character is reached.

Q11.

Username can handle large input

```
=====
  SECURE AUTHENTICATION SYSTEM
=====
Welcome to the secure login system!

Username: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Invalid username!

Username: Invalid username!

Username: Invalid username!
Too many failed attempts. Exiting.
```

Password can handle large input

```
ubuntu@ubuntu:~/CWM-Project/assignment4/buffer_overflow/4B$ ./src/vuln
=====
  SECURE AUTHENTICATION SYSTEM
=====
Welcome to the secure login system!

Username: admin
Password: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Invalid password!
```

Get_user_info() is the vulnerable function as it shows that when a large input is passed to name, it autofills the inputs to the other questions. This is a sign of overflow.


```

=====
SECURE AUTHENTICATION SYSTEM
=====
Welcome to the secure login system!

Username: admin
Invalid username!

Username: secure123
Invalid username!

Username: admin
Password: secure123
Authentication successful!
Enter your name: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Enter your department: Thank you AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA from AAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA!
Enter system command to process: Processing command: AAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Enter response data: Response processed: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
All operations completed successfully.

```

Similar thing is observed for when I input large input into department.

Process_command() is vulnerable too.

When large input is given to “enter system command”, it auto fills the final prompt.

When large input is given to “Enter reponse data”, the output shows segmentation fault.

[illegible]

```
All operations Completed Successfully.
```

```
Exit code: 0  
260          | 0          | OK  
[+] Starting local process './src/vuln': pid 113230  
[+] Receiving all data: Done (285B)  
[*] Process './src/vuln' stopped with exit code -7 (SIGBUS) (pid 113230)  
Response processed: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAA
```

```
Exit code: -7  
264          | -7         | CRASHED! (SIGSEGV)
```

```
[!] Potential offset found at: 264 bytes  
[*] '/home/ubuntu/CWM-Project/assignment4/buffer_overflow/4B/src/vuln'  
Arch:      amd64-64-little  
RELRO:     Partial RELRO  
Stack:     No canary found  
NX:        NX unknown - GNU_STACK missing  
PIE:       No PIE (0x400000)  
Stack:     Executable  
RWX:       Has RWX segments
```

Function Name	Address	Size	Type
process_command	0x4013ec	258	Local
get_user_info	0x401303	233	Local
validate_username	0x4012c9	58	Local
.	0x4012cc	100	Local

SIGSEGV (segmentation fault) occurs at offset 264.

Offset from previous question is 264

The offset of 264 is how many bytes of input are needed to reach the return address on the stack. By setting `target_address` and `offset` in `exploit`, the script crafts an input string which fills the buffer up to byte 264 then writes `win()`'s address at exactly that position.

Q18.

The function `checkvulnerable` compares the password with the secret password character by character. If the character is wrong it returns immediately with no sleep

```
def _vulnerable_check(username: str, password: str) -> bool:
    """
    Intentionally vulnerable password check.
    Iterates character by character and sleeps on each match,
    leaking how many leading characters of the guess are correct.
    """
    if username != SECRET_USERNAME:
        return False

    for i, ch in enumerate(password):
        if i >= len(SECRET_PASSWORD) or ch != SECRET_PASSWORD[i]:
            return False
        time.sleep(DELAY_PER_CHAR)

    return len(password) == len(SECRET_PASSWORD)
```

Q19.

The page content is identical for every failed attempt attacker cannot learn anything from what is displayed. However, the time taken to respond varies based on how many characters are correct. This is a side channel as information leaks through response time.

Attacker can recover the password one character at a time by measuring response times and iterating.

Q20.

Guess	Response time (ms)
0	0
1	0
2	0
3	0
4	0
5	2.1
6	0
7	0
8	0
9	0

Guess 5 is noticeably longer. So, this means first character is 5

Q22.

Guess	Response time (ms)
51	4.3
52	2.2
53	2.2
54	2.1
55	2.3
56	2.2
57	2.2
58	2.2
59	2.1
50	2.1

So second character is 1

Q25.

```
'51926e'      13.5 ms
'51926f'      13.3 ms
'51926g'      13.2 ms
'51926h'      13.3 ms
'51926i'      13.2 ms
'51926j'      13.1 ms
'51926k'      13.2 ms
'51926l'      13.3 ms
'51926m'      13.3 ms
'51926n'      12.9 ms
'51926o'      13.7 ms
'51926p'      13.1 ms
'51926q'      13.3 ms
'51926r'      13.5 ms
'51926s'      13.2 ms
'51926t'      13.1 ms
'51926u'      12.7 ms
'51926v'      13.1 ms
'51926w'      13.7 ms
'51926x'      13.5 ms
'51926y'      13.1 ms
'51926z'      13.6 ms
'519260'      13.2 ms
'519261'      13.3 ms
'519262'      13.3 ms
'519263'      13.4 ms
'519264'      13.6 ms
'519265'      15.3 ms
'519266'      13.3 ms
'519267'      13.3 ms
'519268'      13.5 ms
'519269'      13.1 ms

[+] prefix so far: '519265'

[*] Password cracked: '519265'
<!DOCTYPE html>
<html>
<head><title>Login</title>
```

Q26.

Added a random delay before returning exiting the function.

However, it does not fundamentally eliminate side channel as it only increases the number of measurements an attacker needs to recover the signal through averaging, since random noise cancels out over many samples while the per-character delay does not.

A better fix ensures every code path takes the same number of operations regardless of whether the guess is correct.

Q27.

I used to think that so long as the function is correct and produces the intended result, it is safe to use. I now see that even functions that produce the right output may have vulnerabilities.

The lab shows that information can still be leaked through things like how it accesses memory, how long it takes, what error it produces etc.

When using third party API, i would now not just think about what it returns but look at what side effects it may have.